PUCPR
GRUPO MARISTA

Arquitetura e Soluções Cloud

NexFleet

Sistema Cloud de Gestão e Controle de Frotas

Vinícius dos Reis Góes, Mateus Burlamaqui, Lucas Roussenq e Erik Carvalho

1 - Visão do produto

O NexFleet é uma plataforma web baseada em nuvem destinada ao gerenciamento de frotas empresariais. A solução centralizará os cadastros de veículos e motoristas, os registros de abastecimento e manutenção e a consulta de informações operacionais.

O sistema será desenvolvido para reduzir controles manuais realizados por planilhas, papéis e mensagens dispersas. Ao concentrar essas informações em um único ambiente, o NexFleet facilitará o acompanhamento da situação de cada veículo e dos custos associados à operação.

2 - Problema

Em empresas com vários veículos, motoristas e registros operacionais, as informações podem ficar distribuídas entre documentos, planilhas, notas fiscais e canais de comunicação. Essa fragmentação torna o histórico difícil de consultar e reduz a visibilidade sobre os custos e a disponibilidade da frota.

Como cenário ilustrativo, considera-se uma empresa com 20 veículos, 15 motoristas, vários abastecimentos por semana e manutenções periódicas. A diversidade de quilometragens e vencimentos exige uma forma organizada de controle.

- Manutenções atrasadas e veículos indisponíveis inesperadamente.
- Dificuldade para consolidar gastos com combustível e manutenção.
- Falta de histórico centralizado sobre o uso e a situação dos veículos.
- Baixa visibilidade sobre custos, quilometragem e próximas manutenções.

3 - Objetivo geral

Desenvolver uma plataforma web em ambiente cloud para auxiliar empresas no gerenciamento de suas frotas, permitindo o controle centralizado de veículos, motoristas, abastecimentos, manutenções e informações operacionais.

4 - Objetivos específicos

- Cadastrar e consultar veículos e motoristas.
- Vincular motoristas aos veículos conforme as regras de negócio.
- Registrar abastecimentos, quilometragens e manutenções.
- Consultar o histórico de abastecimentos e manutenções de cada veículo.
- Acompanhar a situação dos veículos, os gastos operacionais e as próximas manutenções.
- Apresentar indicadores iniciais da frota em um dashboard.

5 - Atores do sistema

5.1 - Administrador

O Administrador será responsável pela gestão geral do sistema. Poderá cadastrar usuários, veículos e motoristas, consultar informações, visualizar relatórios e configurar os parâmetros permitidos pela primeira versão.

5.2 - Gestor de Frota

O Gestor de Frota utilizará o NexFleet no dia a dia. Poderá consultar veículos e motoristas, registrar abastecimentos e manutenções e acompanhar custos, históricos e próximas manutenções.

5.3 - Motorista em versões futuras

O Motorista será cadastrado como entidade de negócio no MVP, sem acesso próprio ao sistema. Em uma evolução, poderá registrar abastecimentos e quilometragens, consultar o veículo atribuído e informar problemas operacionais.

6 - Módulos do NexFleet

Módulo	Responsabilidade principal
Usuários e autenticação	Login, cadastro de usuários, perfis e controle de acesso para os papéis iniciais.
Veículos	Cadastro de placa, marca, ano, tipo e status; consulta da quilometragem operacional projetada.
Motoristas	Cadastro de nome, CPF, CNH, categoria, validade, telefone e status.
Abastecimentos	Registro de veículo, motorista, data, combustível, litros, valor, quilometragem e posto.
Manutenções	Registro de tipo, data, quilometragem, descrição, oficina, valor e próxima manutenção.
Dashboard	Visão consolidada de veículos, custos e próximas manutenções.

Os estados inicialmente previstos para os veículos são: disponível, em uso, em manutenção e inativo. Para as manutenções, a classificação inicial contempla: preventiva, corretiva, revisão, troca de óleo, troca de pneus e outros.

7 - Escopo do MVP

O produto mínimo viável concentrará as operações essenciais de gestão de frota. A implementação inicial priorizará cadastros, consultas e registros operacionais, com acesso pelos perfis Administrador e Gestor de Frota.

Área	Entrega da primeira versão
Autenticação	Login, cadastro de usuários e controle de acesso por perfil.
Veículos	Cadastrar, editar, consultar e desativar veículos.
Motoristas	Cadastrar, editar e consultar motoristas.

Área	Entrega da primeira versão
Abastecimentos	Registrar abastecimentos e consultar o histórico.
Manutenções	Registrar manutenções e consultar o histórico.
Dashboard	Consultar a situação da frota, os gastos e as próximas manutenções.

A vinculação entre motoristas e veículos integra os objetivos do produto. O cálculo de consumo médio poderá ser incorporado após a consolidação do histórico. Notificações básicas por evento e função serverless fazem parte da arquitetura; WhatsApp permanece evolução futura.

8 - Funcionalidades futuras

- Acesso próprio para motoristas e aplicativo mobile.
- GPS em tempo real, rotas e integração com rastreadores.
- Controle de multas, documentação e pneus.
- Notificações por WhatsApp e alertas automatizados.
- Integração com postos de combustível.
- Relatórios avançados, consumo médio e custo por quilômetro.
- Previsão de manutenção apoiada por inteligência artificial.

9 - Entidades iniciais

A modelagem inicial contempla cinco entidades principais. Os identificadores, atributos obrigatórios, restrições e cardinalidades serão formalizados no Diagrama de Entidades e Relacionamentos.

Entidade	Responsabilidade
Usuário	Representar a conta de acesso e o perfil de autorização.
Veículo	Armazenar identificação, características e situação; exibir projeção da quilometragem operacional.
Motorista	Armazenar identificação, habilitação, contato e situação do motorista.
Abastecimento	Relacionar veículo e motorista aos dados de cada abastecimento.

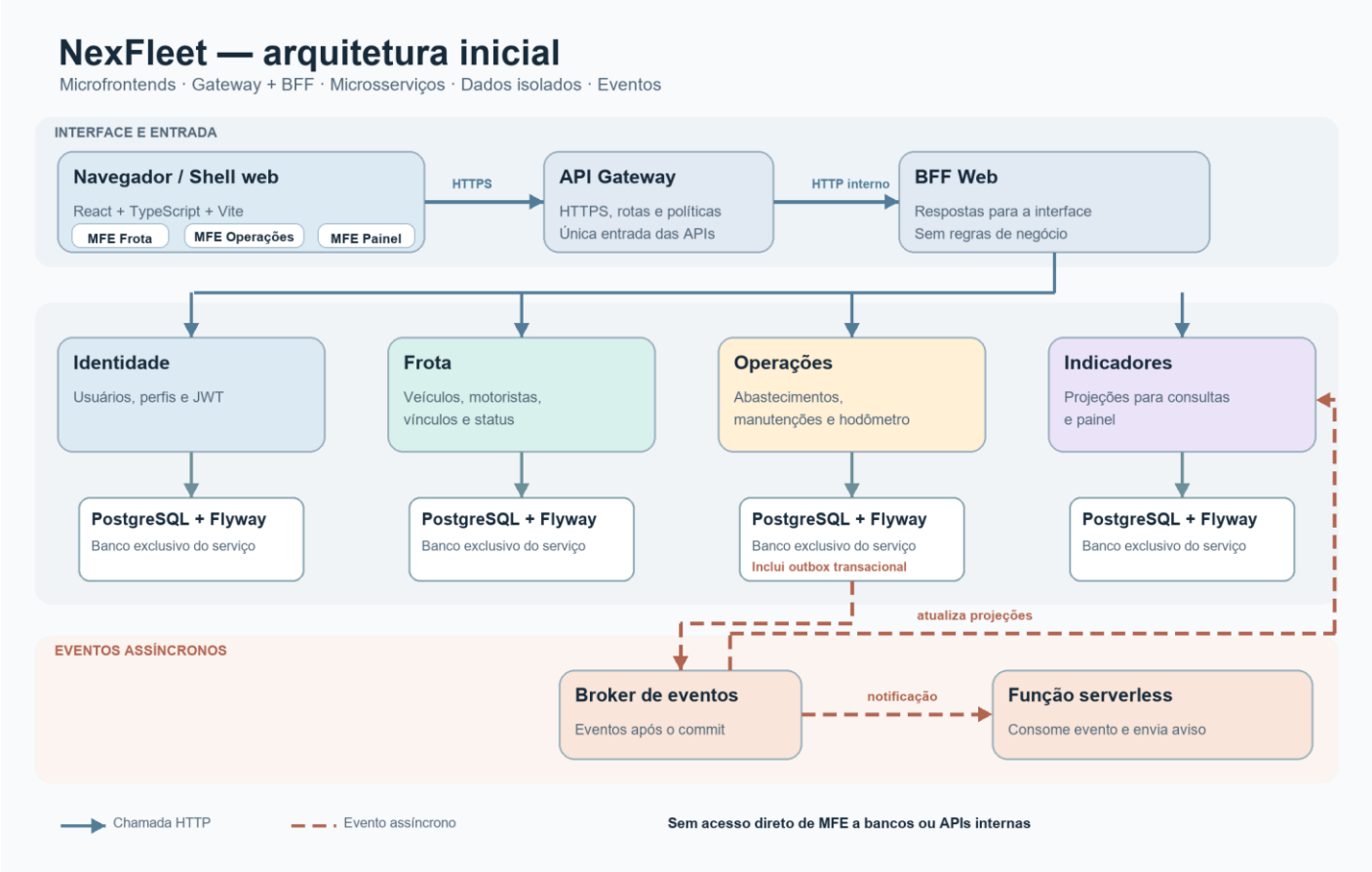
Manutenção	Relacionar o veículo ao serviço realizado, ao custo e à próxima manutenção.

9.1 - Relacionamentos propostos

- Um veículo poderá possuir vários abastecimentos; cada abastecimento estará associado a um veículo.
- Um motorista poderá estar associado a vários abastecimentos; cada abastecimento identificará um motorista.
- Um veículo poderá possuir várias manutenções; cada manutenção pertencerá a um veículo.

A entidade associativa Veiculo - Motorista é uma possibilidade para representar a atribuição de motoristas a veículos e registrar os períodos de vínculo.

10 - Arquitetura inicial



11 - Stack

A implementação usa React, TypeScript e Vite; Java 21, Spring Boot, Spring Security, JWT e BCrypt; PostgreSQL com Flyway; Docker Compose no ambiente local; RabbitMQ para eventos locais. A publicação futura mapeia gateway, serviços, broker e funções à Azure.

Camada	Tecnologia proposta	Serviço cloud proposto
Frontend	React + TypeScript + Vite; shell e microfrontends	Azure Static Web Apps

Backend e API	Java 21 + Spring Boot; BFF e microsserviços REST	Azure Container Apps
Banco de dados	PostgreSQL por microsserviço + Flyway	Azure Database for PostgreSQL, bases isoladas
Empacotamento	Docker Compose local	Conforme a estratégia de implantação
Gateway, eventos e funções	API Gateway, RabbitMQ e worker serverless	API Management, Service Bus e Azure Functions
Observabilidade	Logs, métricas e correlação	Azure Monitor

12 Primeiro fluxo de negócio

12.1 - Registro de abastecimento

O registro de abastecimento atravessa o microfrontend Operações, o API Gateway e o BFF até o serviço Operações. Esse serviço consulta o contrato público de Frota para validar veículo e motorista ativos, grava abastecimento, hodômetro e outbox em uma transação do seu próprio banco e publica o evento após o commit.

Pré-condições propostas: usuário autenticado e autorizado, veículo e motorista previamente cadastrados.

1. O gestor acessa o módulo de abastecimentos e inicia um novo registro.
2. Seleciona o veículo e identifica o motorista associado.
3. Informa data, combustível, quantidade em litros, valor, quilometragem e posto.
4. O microfrontend envia os dados ao BFF por HTTPS através do API Gateway.
5. O BFF encaminha o comando à API de Operações, que valida campos, autorização e referências por contrato público de Frota.
6. Operações grava abastecimento, hodômetro operacional e intenção de publicação no outbox em uma transação local.
7. O publicador envia AbastecimentoRegistrado ao broker; consumidores atualizam suas projeções de forma idempotente.
8. O serviço confirma o registro. O dashboard pode refletir o evento depois de pequeno atraso, indicado ao usuário quando necessário.

12.2 - Validações propostas

A API deverá verificar os campos obrigatórios, a existência das referências e a consistência de litros, valor e quilometragem. As regras para lançamentos retroativos e correções do hodômetro serão definidas antes da implementação.

Abastecimento, hodômetro operacional e outbox são gravados na mesma transação do banco de Operações. A atualização de projeções em outros serviços é assíncrona e não usa transação distribuída. Falhas de publicação são repetidas; mensagens persistentes não processadas vão para fila de erro.

Resultado esperado: registro e hodômetro consistentes em Operações; histórico e indicadores atualizados por evento. O diagrama UML de sequência da Parte 2 representa esse fluxo.

13 Requisitos do sistema

Os requisitos do NexFleet foram definidos a partir do escopo do MVP. Eles estabelecem os serviços que a aplicação deverá oferecer e as condições de qualidade que deverão orientar a implementação.

13.1 - Requisitos não funcionais

Código	Requisito não funcional
RNF01	O sistema deve permitir acesso por navegador em computadores e dispositivos móveis.
RNF02	O sistema deve utilizar HTTPS na comunicação entre o frontend e a API.
RNF03	O sistema deve armazenar senhas de forma segura por meio de hash.
RNF04	O sistema deve restringir as operações da API conforme o perfil autenticado.
RNF05	O sistema deve utilizar banco de dados relacional para os dados operacionais.
RNF06	O sistema deve preservar o histórico válido de abastecimentos e manutenções.
RNF07	O sistema deve apresentar telas claras e adequadas a usuários não técnicos.
RNF08	O backend deve usar BFF e microsserviços com banco por serviço; microfrontends, gateway, EDA e função serverless são partes da arquitetura.
RNF09	O sistema deve permitir implantação em ambiente cloud.

13.2 - Requisitos funcionais

Código	Requisito funcional

RF01	O sistema deve permitir que usuários façam login no sistema.
RF02	O sistema deve permitir ao Administrador cadastrar, editar, consultar e desativar usuários.
RF03	O sistema deve controlar o acesso conforme os perfis Administrador e Gestor de Frota.
RF04	O sistema deve cadastrar veículos com placa, marca, modelo, ano, tipo, quilometragem, status e data de aquisição.
RF05	O sistema deve editar, consultar e desativar veículos.
RF06	O sistema deve cadastrar motoristas com nome, CPF, CNH, categoria, validade, telefone e status.
RF07	O sistema deve editar e consultar motoristas.
RF08	O sistema deve vincular um motorista a um veículo e preservar o histórico do vínculo.
RF09	O sistema deve registrar abastecimentos com veículo, motorista, data, combustível, litros, valor, quilometragem e posto.
RF10	O sistema deve atualizar a quilometragem do veículo após um abastecimento válido.
RF11	O sistema deve consultar o histórico de abastecimentos por veículo.
RF12	O sistema deve registrar manutenções com veículo, tipo, data, quilometragem, descrição, oficina, valor e próxima manutenção.
RF13	O sistema deve consultar o histórico de manutenções por veículo.
RF14	O sistema deve exibir no dashboard os principais indicadores operacionais e financeiros da frota.
RF15	O sistema deve consultar veículos de acordo com seu status.

14 - Regras de negócio

As regras de negócio definem as validações e restrições que garantem a consistência das operações do NexFleet.

Código	Regra de negócio
RB01	Cada usuário deve possuir o perfil Administrador ou Gestor de Frota.
RB02	Somente usuários autenticados podem acessar as funcionalidades do sistema.
RB03	A placa de cada veículo deve ser única.
RB04	O CPF e o número da CNH de cada motorista devem ser únicos.
RB05	Veículos inativos não podem receber novos abastecimentos, manutenções ou vínculos.
RB06	No MVP, cada veículo pode possuir somente um vínculo ativo com motorista por vez.
RB07	O motorista vinculado deve estar ativo e possuir CNH dentro da validade.
RB08	O abastecimento deve conter os campos obrigatórios definidos no requisito RF09.
RB09	Litros, valor total e quilometragem devem ser maiores que zero.
RB10	A quilometragem informada não pode ser inferior à quilometragem atual do veículo.
RB11	Um abastecimento válido deve atualizar a quilometragem atual do veículo.
RB12	A manutenção deve conter os dados obrigatórios definidos no requisito RF12.
RB13	A próxima manutenção, quando informada, deve ser posterior à manutenção registrada.
RB14	Abastecimentos e manutenções válidos devem ser preservados no histórico.
RB15	Os gastos do dashboard devem ser calculados a partir dos registros válidos.
RB16	Veículos em manutenção não devem ser contabilizados como disponíveis.

15 - Diagrama de Entidades e Relacionamentos

Entidade conceitual	Proprietário no DER físico

Usuário	Identidade
Veículo, motorista e vínculo	Frota
Abastecimento, manutenção e hodômetro	Operações
Indicadores agregados	Indicadores

Restrições verificadas pelo serviço proprietário

Regra	Fonte da verdade e verificação
Placa única	Frota: índice único em veículo.
CPF e CNH únicos	Frota: índices únicos em motorista.
Vínculo ativo único	Frota: regra e restrição para veículo ativo.
Hodômetro não regressivo	Operações: transação local no registro.
Evento processado uma vez	Indicadores: eventId idempotente.

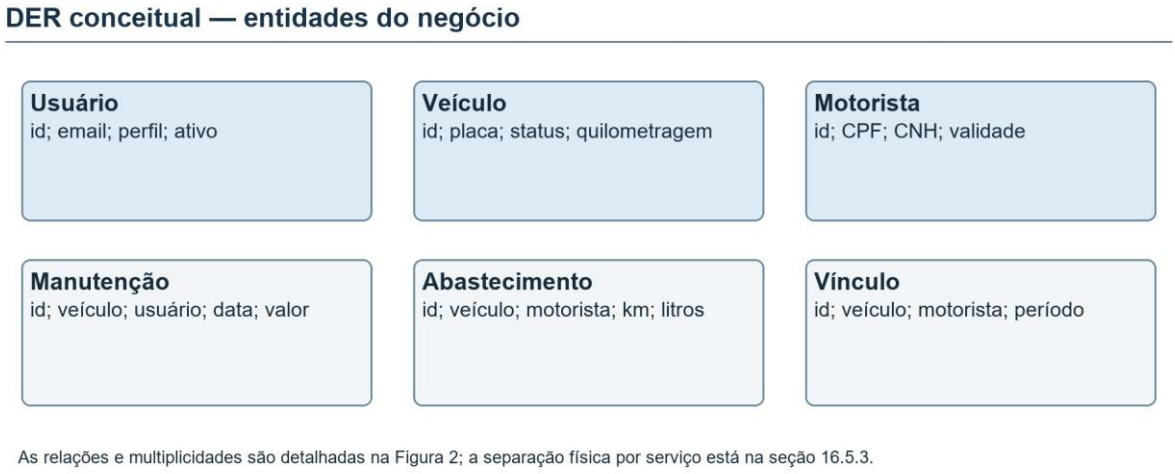


Figura 1 - Diagrama de Entidades e Relacionamentos do NexFleet



Figura 2 - Detalhamento das cardinalidades do DER



Figura 3 - Enumerações utilizadas pelo modelo de dados

16 - Documentação Arc42 da arquitetura

16.1 - Introdução e metas

O NexFleet apoia empresas no cadastro de frota, na operação diária e na análise de custos. Administrador e Gestor de Frota são os usuários iniciais. A arquitetura deve demonstrar distribuição de responsabilidades, implantação independente e rastreabilidade de decisões, sem perder a coerência das regras de negócio.

Meta	Evidência esperada
Segurança	Autenticação, autorização por perfil, hash BCrypt, JWT validado e segredos externos.
Consistência	Transação local em Operações; entrega confiável por outbox e consumidores idempotentes.
Evolução	Microfrontends e microsserviços publicáveis independentemente; contratos versionados.
Operabilidade	Health checks, correlationId, logs, métricas e filas de erro observáveis.

Requisitos	Área	Resultado previsto
RF01–RF03	Acesso e usuários	Login, administração de contas e perfis Administrador/Gestor.
RF04–RF05	Veículos	Cadastrar, editar, consultar e desativar, com situação e km projetado.
RF06–RF07	Motoristas	Cadastrar, editar e consultar CNH, validade e situação.
RF08	Vínculos	Associar motorista e veículo preservando períodos históricos.
RF09–RF11	Abastecimentos	Registrar, validar hodômetro e consultar histórico.
RF12–RF13	Manutenções	Registrar custos, km, próxima data e histórico.

RF14–RF15	Painel	Consultar disponibilidade e indicadores de frota.

Parte interessada	Expectativa
Administrador	Gerenciar contas, perfis e cadastros com acesso controlado.
Gestor de Frota	Registrar e consultar operações com clareza e consistência.
Equipe de quatro alunos	Construir e demonstrar arquitetura, código, testes e documentação coerentes.
Professor e avaliadores	Verificar estilos exigidos, diagramas legíveis e rastreabilidade.
Motorista (evolução)	Registrar ou consultar dados de veículo atribuído futuramente.

Metas priorizadas: (1) segurança — acesso indevido recebe 401/403 sem vazar dados; (2) consistência e rastreabilidade — km inválido não é gravado e o responsável fica registrado; (3) usabilidade — formulários mostram erros por campo e confirmação compreensível.

16.2 - Restrições

ID	Restrição	Consequência
R1	React + TypeScript + Vite	Shell e microfrontends por domínio, com contratos de integração definidos.
R2	Java 21 + Spring Boot + API REST	BFF e APIs de domínio implantáveis separadamente.
R3	PostgreSQL + Flyway	Base e histórico de migrações próprios por serviço.
R4	Todos os estilos exigidos	Gateway, BFF, microsserviços, EDA, serverless e testes arquiteturais integram o escopo.
R5	Docker Compose local e Azure futura	Paridade de contratos entre ambientes; publicação cloud depende de provisionamento.

Restrições Organizacionais

ID	Restrição organizacional	Impacto arquitetural
RO01	Uso empresarial e evolução em cloud.	Contratos estáveis, implantação separada e observabilidade.
RO02	LGPD, segurança e rastreabilidade.	Minimização de dados, autorização, auditoria e retenção.

RO03	Mudanças avaliadas por impacto, custo e risco.	ADRs e backlog antes de alterações significativas.
RO04	Primeira entrega funcional limitada ao MVP.	Estilos exigidos em implementação mínima; anexos/WhatsApp depois.
RO05	Documentação C4, UML, Canvas e Arc42.	Diagramas e texto revistos com o código em cada sprint.
ID	**Restrição técnica**	**Aplicação**
RT01	React + TypeScript + Vite.	Shell e microfrontends de domínio.
RT02	Java 21 + Spring Boot + REST.	Gateway, BFF e APIs próprias.
RT03	PostgreSQL + Flyway.	Banco e histórico de migração por serviço.
RT04	Azure como publicação futura.	Correspondência de recursos após orçamento e autorização.
RT05	Estilos distribuídos exigidos.	Microservices/DB per Service, EDA, serverless e testes de fronteira.
RT06	HTTPS e JSON.	Comunicação pública segura e contratos versionados.

16.3 - Contexto e escopo

No contexto de negócio, Administrador gerencia contas e permissões, o Gestor registra operações e consulta a frota. O serviço externo de e-mail recebe solicitações de notificação. O Azure Monitor recebe telemetria técnica, sem acesso direto aos dados transacionais.

Dentro do escopo	**Fora do escopo inicial**
Usuários, perfis, veículos, motoristas e vínculos.	Integração com ERP, rastreador GPS ou pagamento.
Abastecimentos, manutenções, histórico e indicadores.	WhatsApp e anexos de comprovantes na primeira versão.
Avisos por evento e observabilidade operacional.	Publicação Azure antes da aprovação de custos e segurança.

C4 Nível 1 — Contexto do NexFleet

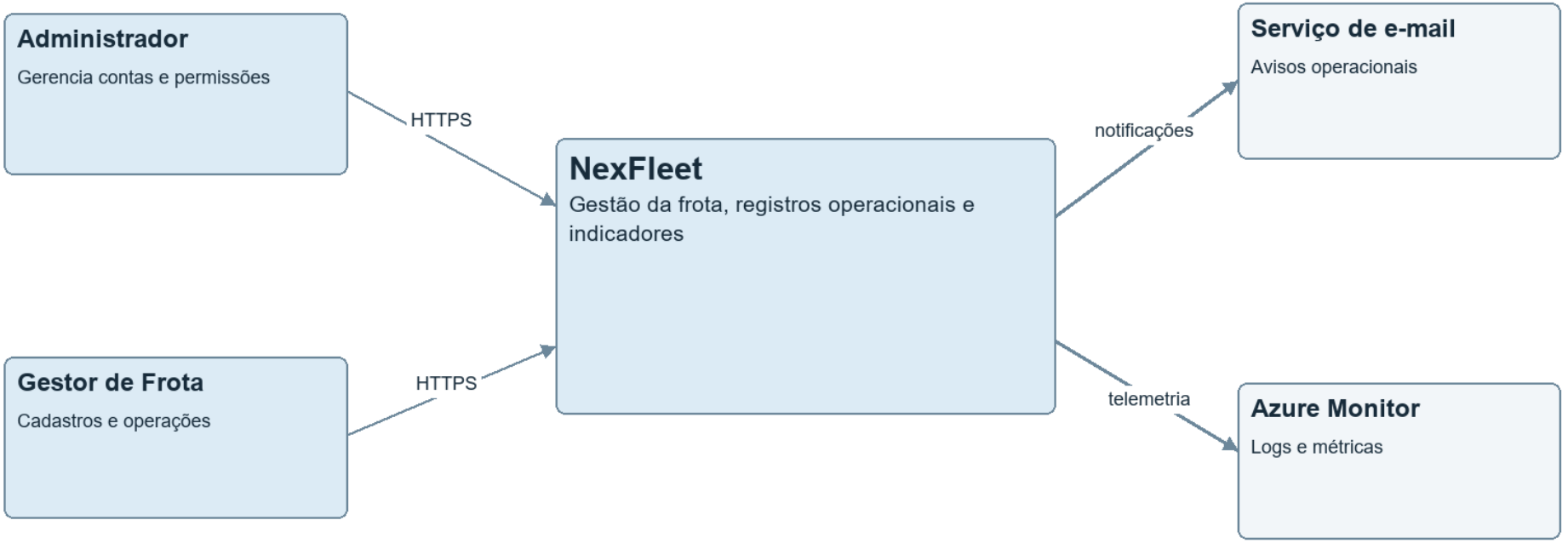


Figura 4 - C4 Nível 1: pessoas, NexFleet e sistemas externos.

Origem	Destino	Informação / canal
Administrador/Gestor	Shell + MFEs	Credenciais, cadastros, operações e consultas por HTTPS.
Microfrontends	Gateway → BFF	JSON por HTTPS; sem acesso direto ao banco.
BFF	APIs Identidade/Frota/Operações/Indicadores	HTTP interno autenticado, tempo limite e correlação.
Cada serviço	PostgreSQL próprio	SQL e transações apenas em sua base.
Outbox	Broker	Eventos versionados e persistentes.
Função serverless	E-mail externo	Avisos mínimos; falhas observáveis.
Todos os processos	Monitor	Métricas, traces e logs sem dados pessoais completos.
Evolução fora do MVP	Blob Storage	Comprovantes, somente após decisão e controle de acesso.

16.4 - Estratégia da solução

O shell web carrega microfrontends de Frota, Operações e Painel. O API Gateway é a única entrada pública das APIs. O BFF compõe respostas adequadas à interface, mas não contém regras de domínio. Identidade, Frota, Operações e Indicadores são microsserviços com implantação e bancos próprios. Eventos transportam fatos concluídos e a função serverless envia avisos sem participar da transação de negócio.

Elemento	O que faz	O que não faz
Microfrontend	Entrega uma área da interface; publica rotas e componentes.	Não acessa banco ou API interna sem gateway.
API Gateway	Expõe rotas, aplica TLS, limites e observabilidade de entrada.	Não implementa regra de abastecimento.
BFF	Adapta e agrega dados para a tela web.	Não compartilha banco com serviços.
Broker	Distribui eventos persistidos no outbox.	Não substitui a fonte da verdade.
Função serverless	Consome evento e envia aviso; escala por demanda.	Não decide regras transacionais.

A autonomia prometida pelos microsserviços só existe se cada um puder migrar e consultar seu próprio banco sem coordenação de SQL com outro domínio. Chamadas síncronas ficam
restritas à validação indispensável no comando, já painéis, avisos e agregações podem aceitar atualização eventual. O gateway controla a borda, enquanto o BFF prepara a resposta para a tela sem assumir regras da operação.

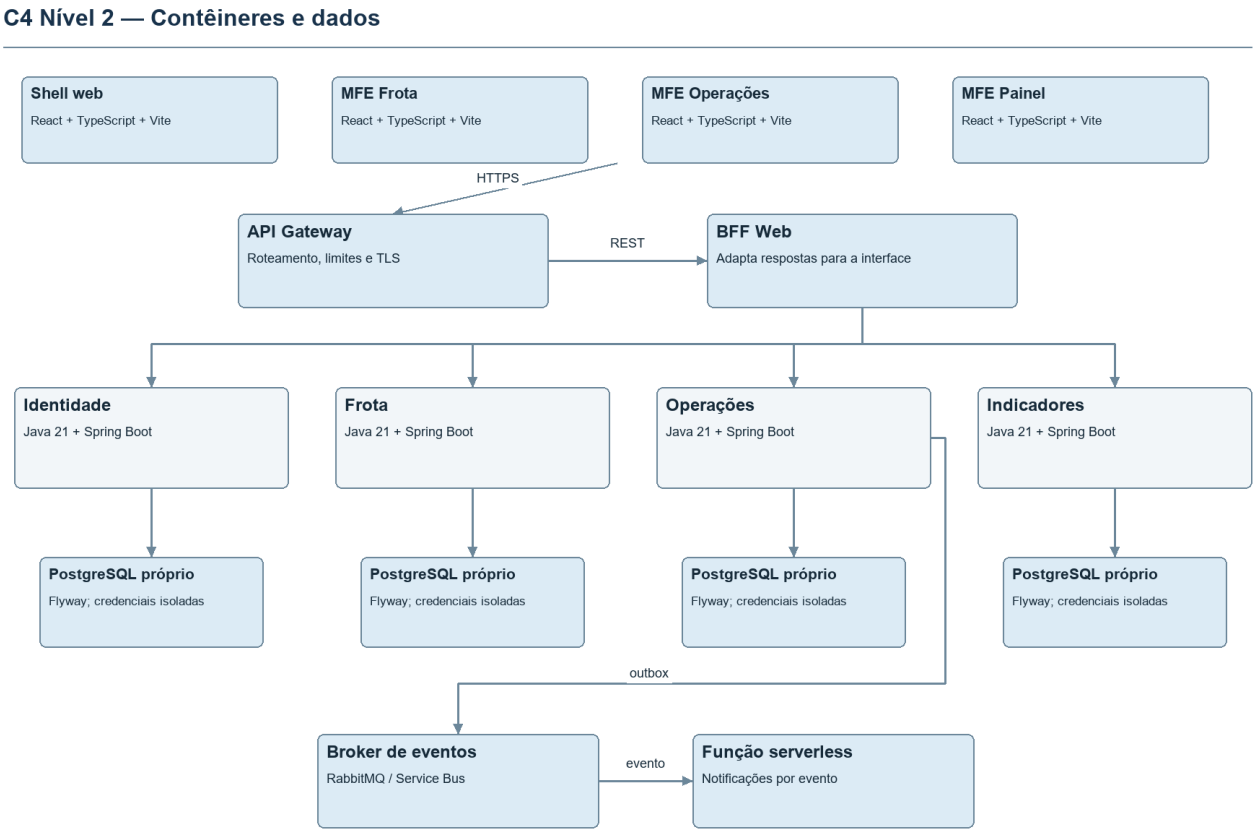
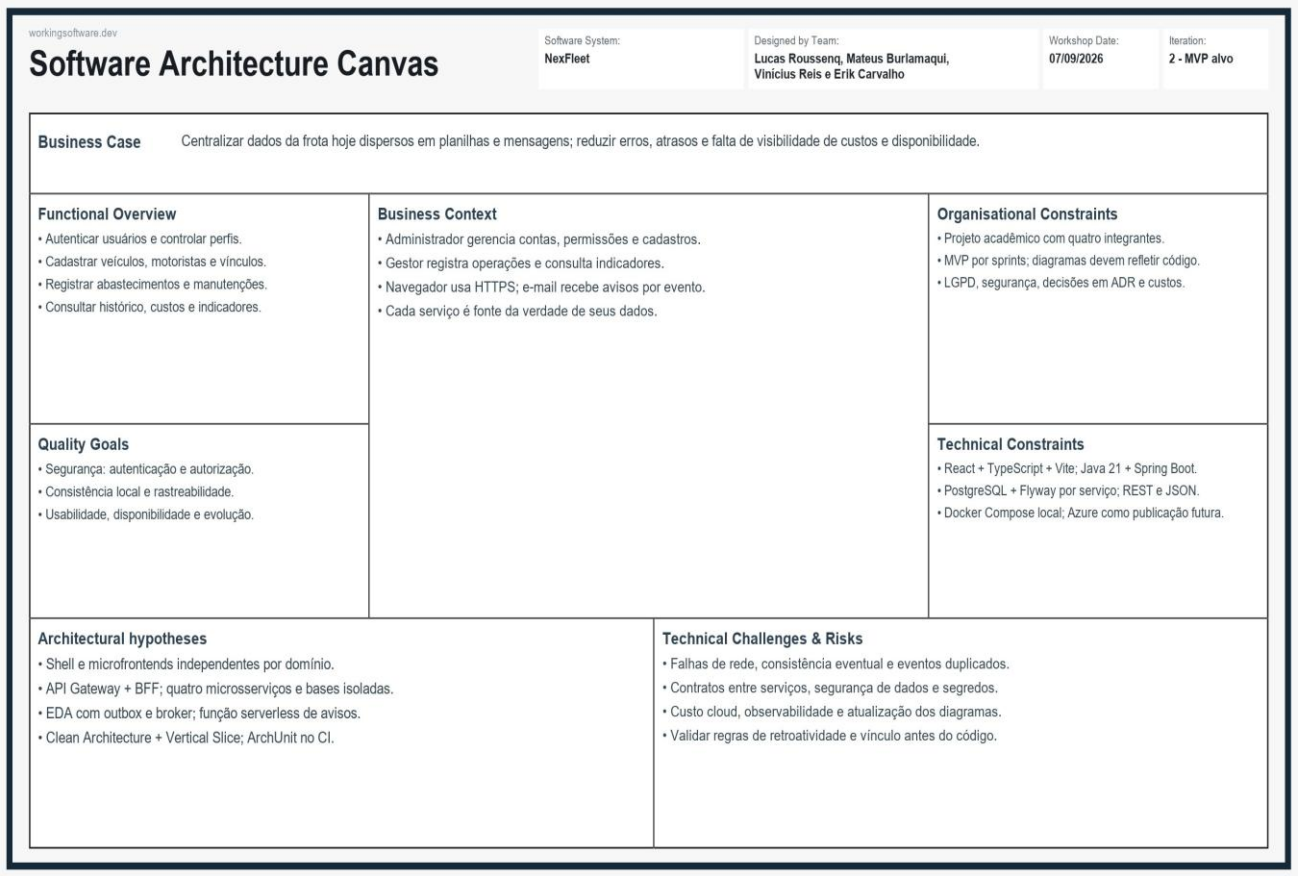


Figura 5 - C4 Nível 2: contêineres, interfaces e bancos por serviço.



workingsoftware.dev

Software Architecture Canvas

Software System:	Designed by Team:	Workshop Date:	Iteration:
NexFleet	**Lucas Rousseng, Mateus Burlamaqui, Vinicius Reis e Erik Carvalho**	**07/09/2026**	**2 - MVP alvo**

Business Case Centralizar dados da frota hoje dispersos em planilhas e mensagens; reduzir erros, atrasos e falta de visibilidade de custos e disponibilidade.

Functional Overview
- Autenticar usuários e controlar perfis.
- Cadastrar veículos, motoristas e vínculos.
- Registrar abastecimentos e manutenções.
- Consultar histórico, custos e indicadores.

Business Context
- Administrador gerencia contas, permissões e cadastros.
- Gestor registra operações e consulta indicadores.
- Navegador usa HTTPS; e-mail recebe avisos por evento.
- Cada serviço é fonte da verdade de seus dados.

Organisational Constraints
- Projeto acadêmico com quatro integrantes.
- MVP por sprints; diagramas devem refletir código.
- LGPD, segurança, decisões em ADR e custos.

Quality Goals
- Segurança: autenticação e autorização.
- Consistência local e rastreabilidade.
- Usabilidade, disponibilidade e evolução.

Technical Constraints
- React + TypeScript + Vite; Java 21 + Spring Boot.
- PostgreSQL + Flyway por serviço; REST e JSON.
- Docker Compose local; Azure como publicação futura.

Architectural hypotheses
- Shell e microfrontends independentes por domínio.
- API Gateway + BFF; quatro microsserviços e bases isoladas.
- EDA com outbox e broker; função serverless de avisos.
- Clean Architecture + Vertical Slice; ArchUnit no CI.

Technical Challenges & Risks
- Falhas de rede, consistência eventual e eventos duplicados.
- Contratos entre serviços, segurança de dados e segredos.
- Custo cloud, observabilidade e atualização dos diagramas.
- Validar regras de retroatividade e vínculo antes do código.

Figura 6 - Software Architecture Canvas do NexFleet.

BACHARELADO EM ENGENHARIA DE SOFTWARE
Somativa Arquitetura de Soluções com Cloud

16.4.1 - Software Architecture Canvas

O canvas reproduz o template visual da primeira entrega parcial: identificação no topo; Business Case em toda a largura; Functional Overview e Quality Goals à esquerda; Business Context ao centro; Organisational e Technical Constraints à direita; hipóteses e riscos na base. Os campos foram atualizados para a arquitetura-alvo do MVP.

Campo do canvas	Definição NexFleet
Problema	Dados dispersos, histórico difícil e pouca visibilidade de custos e disponibilidade.
Usuários e valor	Administrador governa acesso; Gestor executa operações e consulta indicadores.
Capacidades	Identidade, Frota, Operações, Indicadores e notificações.
Qualidade	Segurança, consistência local, evolução independente e observabilidade.
Decisões	Microfrontends; gateway + BFF; microsserviços com base própria; EDA; função serverless.
Riscos	Custo operacional, falhas de rede, consistência eventual e contratos entre equipes.
Validação	Testes de arquitetura, integração, contratos, fluxo ponta a ponta e cenários de falha.

16.4.2 - Aplicação dos estilos exigidos

Estilo	Onde aparece	Critério de implementação
Microfrontend	Shell, Frota, Operações e Painel.	Artefatos independentes; rotas e design system compartilhados por contrato.
API Gateway	Entrada das APIs; Spring Cloud Gateway local e API Management na Azure.	Roteamento, políticas de borda, TLS e identificação de correlação.
BFF	BFF Web entre gateway e APIs.	Composição para telas sem acesso a bancos ou regras de domínio.

Microservices + Database per Service	Identidade, Frota, Operações e Indicadores.	Deploy, credenciais, esquema e Flyway próprios; sem SQL cruzado.
EDA	Outbox, RabbitMQ/Service Bus e consumidores.	Eventos versionados, idempotência, retentativas e DLQ.
Serverless	Função NotificarAbastecimento.	Gatilho por evento, execução sob demanda e idempotência.
Clean + Vertical Slice	Dentro de BFF e cada serviço.	Pacotes por caso de uso; dependência para o domínio.
Testes de arquitetura	CI Java e frontend.	ArchUnit e regras de importação quebram o build em violações.

16.5 - Visão de construção

O C4 Nível 3 abre o serviço Operações. A API HTTP traduz requisições; a fatia Registrar Abastecimento coordena o caso de uso; o domínio protege invariantes; portas abstraem persistência e publicação; adaptadores implementam detalhes técnicos. O serviço Identidade emite JWT; Frota possui veículo, motorista e vínculo; Indicadores mantém projeções de leitura.

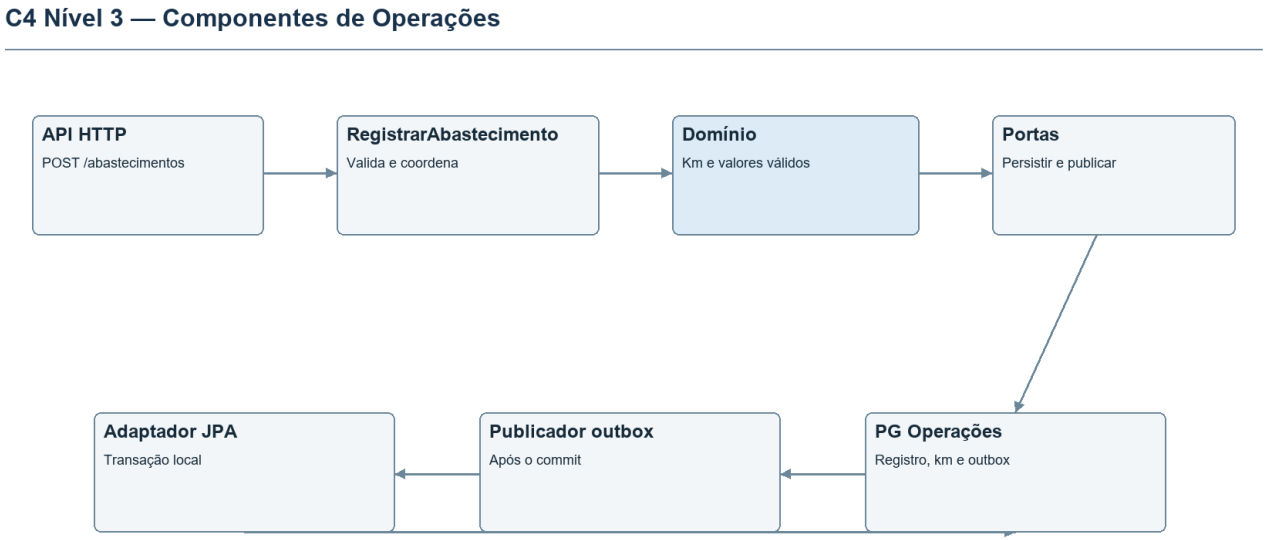


Figura 7 - C4 Nível 3: componentes internos de Operações.

Serviço ou componente	Responsabilidade	Colaboração permitida
Identidade	Conta, perfil, senha e emissão de JWT.	API pública de identidade; publica alterações relevantes.
Frota	Veículos, motoristas, status e vínculos históricos.	Publica mudanças para projeções; não altera Operações.
Operações	Abastecimentos, manutenções, hodômetro e outbox.	Valida comandos na API pública de Frota; publica fatos.
Indicadores	Projeções de custo e disponibilidade.	Consome eventos; não escreve nas fontes de verdade.
Gateway	Políticas de entrada e roteamento.	Invoca BFF/APIs públicas; sem regra de negócio.
BFF	Resposta específica da interface.	Agrega APIs; não compartilha banco.
Observabilidade	Logs, métricas e correlação.	Transversal sem coletar segredos.

16.5.1 - Clean Architecture e Vertical Slice

Clean Architecture é a regra de dependência: domínio não importa Spring, JPA ou HTTP; aplicação usa entidades e portas; adaptadores externos dependem dessas abstrações. Vertical Slice organiza o código por caso de uso completo, por exemplo registrarAbastecimento, consultarHistorico e registrarManutencao, em vez de um único pacote global de controllers ou services.

Caminho de código previsto	Responsabilidade
operacoes/registrarabastecimento/api	Controller e DTO de entrada/saída.
operacoes/registrarabastecimento/application	Comando, caso de uso e portas.
operacoes/registrarabastecimento/domain	Entidades, invariantes e erros de negócio.
operacoes/registrarabastecimento/infrastructure	JPA, outbox e clientes de contratos externos.

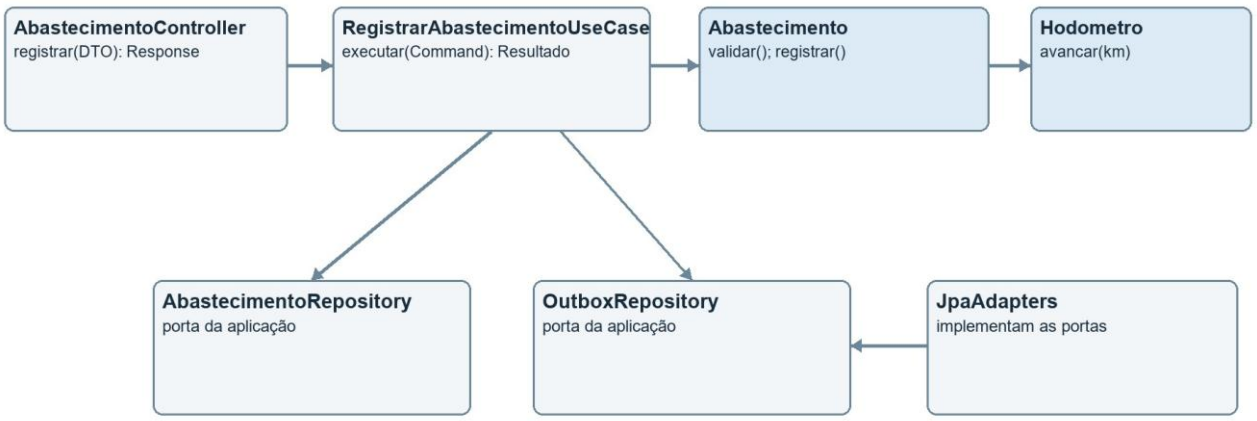
Essa estrutura é contrato para o código a entregar. A regra verificável é api → application → domain, enquanto infrastructure implementa portas; dependências em sentido inverso são proibidas.

Exemplo de outras fatias: frota/cadastrarveiculo, frota/vincularmotorista, operacoes/registrarmanutencao e indicadores/consultardashboard. Cada fatia reúne seus testes de domínio, aplicação e API. Tipos compartilhados devem ser mínimos; serviços não importam pacotes internos de outros serviços.

Fronteira verificável	Exemplo de teste arquitetural
Domínio não conhece framework.	Falha se domain importar Spring, JPA ou classe web.
Aplicação depende de portas, não adaptadores.	Falha se use case importar JpaRepository concreto.
Fatia é coesa e serviços não compartilham internos.	Falha se uma fatia importar implementation de outra API.

16.5.2 - C4 Nível 4 e UML

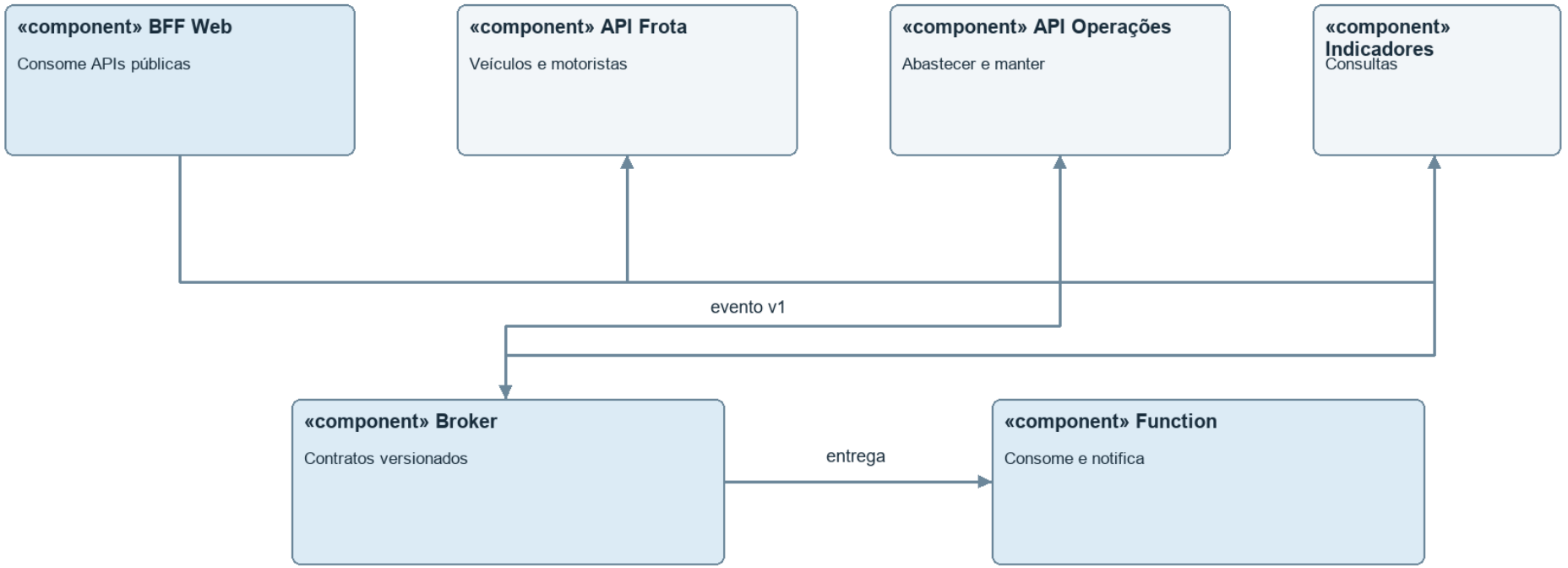
C4 Nível 4 — Fatia Registrar Abastecimento



Transação local: abastecimento + hodômetro + outbox no banco de Operações.

Figura 8 - C4 Nível 4: classes e dependências da fatia Registrar Abastecimento.

UML de componentes — Contratos entre serviços



Cada API possui implantação e banco próprios; eventos não expõem tabelas internas.

Figura 9 - UML de componentes: APIs, broker e função.

O C4 Nível 4 é complementado por UML de componentes e classes nesta seção e pelas três sequências da seção 16.6, onde a ordem das chamadas e o rollback ficam visíveis. Notação UML: “−” indica atributo privado e “+” operação pública; 1 exige uma ocorrência e 0..N admite nenhuma ou várias. Controller e adaptadores são detalhes externos; Abastecimento e Hodometro concentram invariantes, e os repositórios são portas da aplicação.

UML de classes — Caso de uso de abastecimento

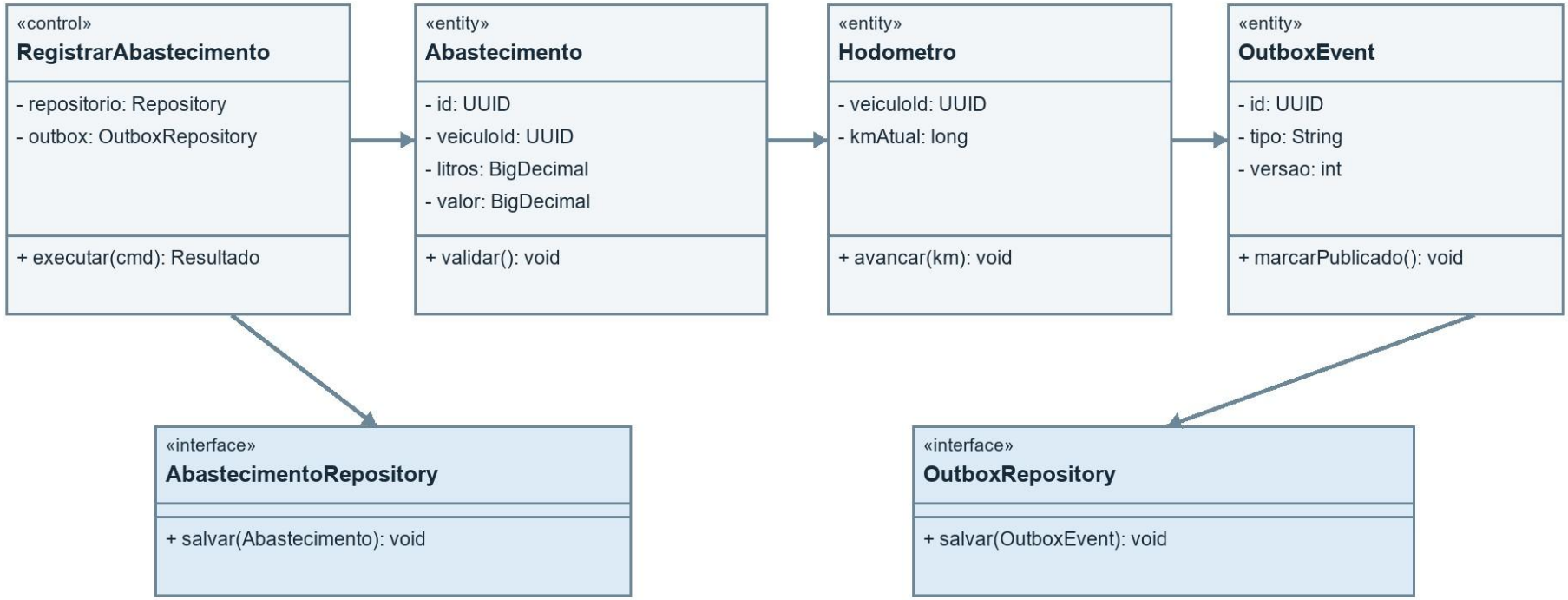


Figura 10 - UML de classes: contratos do caso de uso crítico.

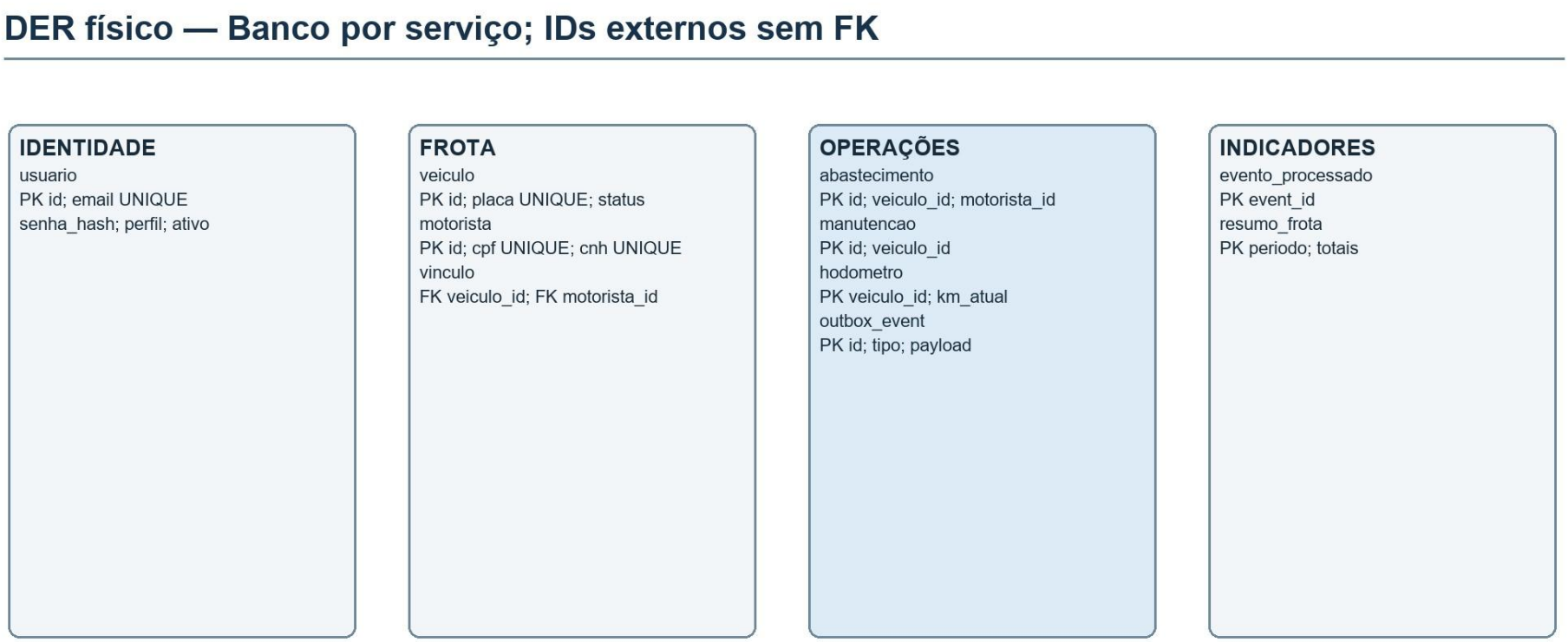
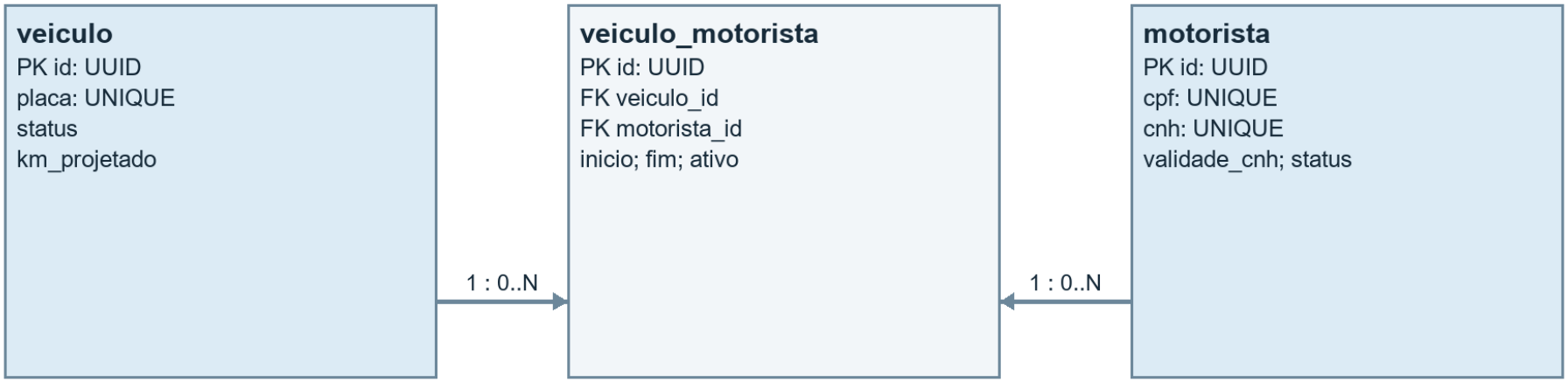


Figura 11 - DER físico distribuído por serviço.

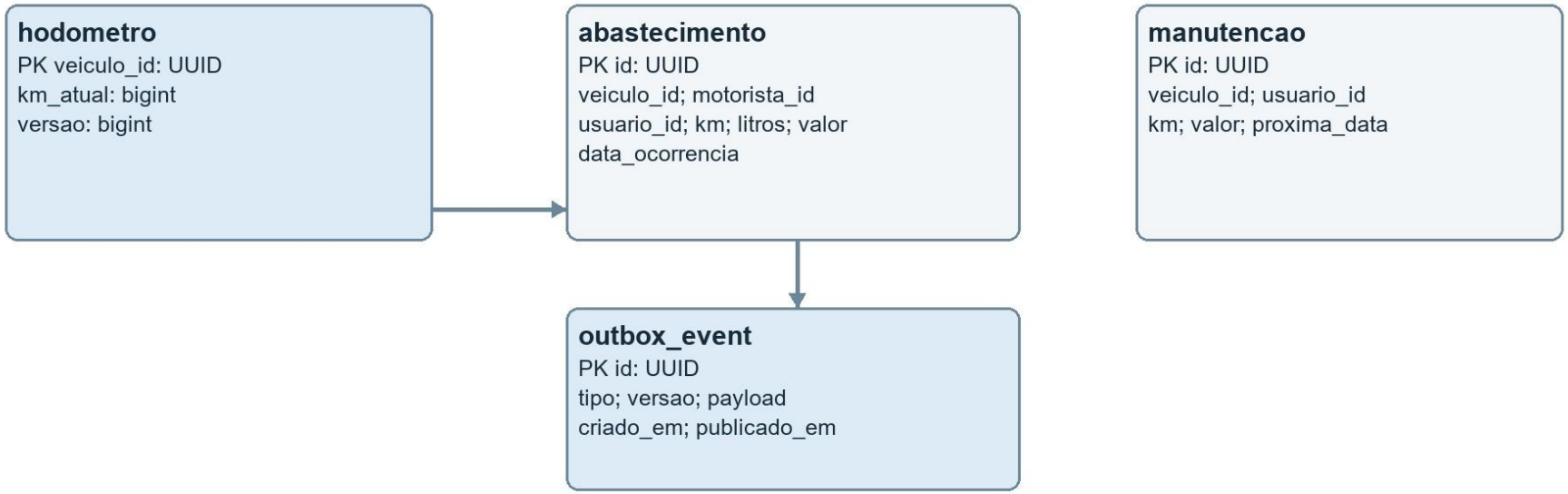
DER Frota — chaves e cardinalidades



Todas as FKs deste desenho pertencem ao banco Frota.

Figura 12 - DER da Frota: veículo, motorista e vínculo histórico.

DER Operações — transação e outbox



IDs de Frota e Identidade são referências externas sem FK; não há transação entre bancos.

Figura 13 - DER de Operações: registro, hodômetro e outbox.

16.5.3 - DER e propriedade dos dados

Os diagramas anteriores detalham o DER conceitual da Parte 1 em bancos físicos separados. Identidade possui usuário; Frota possui veículo, motorista e vínculo; Operações possui abastecimento, manutenção, hodômetro e outbox; Indicadores possui projeções e IDs de eventos processados. Referências entre serviços são UUIDs sem FK cruzada.

Relação dentro do banco proprietário	**Cardinalidade**
Frota: veículo → vínculo de motorista	1 para 0..N vínculos históricos.
Frota: motorista → vínculo de veículo	1 para 0..N vínculos históricos.
Operações: veículoId → abastecimento	1 referência para 0..N registros, sem FK cruzada.
Operações: veículoId → manutenção	1 referência para 0..N registros, sem FK cruzada.

Entidade física	**Chave e campos principais**	**Restrição ou relação**
Identidade.usuario	id UUID; email; senha_hash; perfil; ativo.	email UNIQUE; senha em hash BCrypt.
Frota.veiculo	id UUID; placa; status; km_projetado.	placa UNIQUE; km projetado por evento.
Frota.motorista	id UUID; cpf; cnh; validade_cnh; status.	cpf e cnh UNIQUE; dados protegidos.
Frota.veiculo_motorista	id UUID; veiculo_id; motorista_id; inicio; fim.	FKs locais; histórico e vínculo ativo controlado.
Operações.abastecimento	id UUID; veiculo_id; motorista_id; usuario_id; km; litros; valor.	IDs externos sem FK; valores positivos.
Operações.manutencao	id UUID; veiculo_id; usuario_id; km; valor; proxima_data.	IDs externos sem FK; histórico preservado.
Operações.hodometro	veiculo_id UUID; km_atual; versao.	Controle concorrente; km não regressivo.
Operações.outbox_event	id UUID; tipo; versao; payload; publicado_em.	Gravado na mesma transação do registro.
Indicadores.evento_processado	event_id UUID; processado_em.	Unicidade garante idempotência.
Indicadores.resumo_frota	periodo; totais; atualizado_em.	Projeção reconstruível a partir de eventos.

16.5.4 - Rastreabilidade de requisitos e artefatos

Requisito	**Regra**	**Fatia proprietária**	**Prova no código**
RF01–RF03	RB01–RB02	Identidade/login e gerenciarUsuario.	401/403, hash e teste de autorização.
RF04–RF08	RB03–RB07	Frota/cadastrarVeiculo, cadastrarMotorista, vincular.	Domínio, índices únicos e integração.
RF09–RF11	RB08–RB11	Operações/registrarAbastecimento.	Rollback, concorrência, outbox e sequência.
RF12–RF13	RB12–RB14	Operações/registrarManutencao.	Datas, custos e histórico.
RF14–RF15	RB15–RB16	Indicadores/consultarPainel.	Agregações e disponibilidade por projeção.

Cadeia exemplar: RF09–RF11 → RB08–RB11 → RegistrarAbastecimentoUseCase → sequência da seção 16.6 → testes de domínio, integração e arquitetura. O C4 Nível 1 explica

atores; o Nível 2 explica implantação; o Nível 3 explica componentes; o Nível 4 e UML explicam o código crítico; o DER explica propriedade dos dados.

16.6 - Visão de execução

O usuário autentica-se pelo BFF/Identidade. A senha é comparada ao hash BCrypt; Identidade emite JWT de curta duração com perfil. Gateway e serviços verificam o token conforme sua

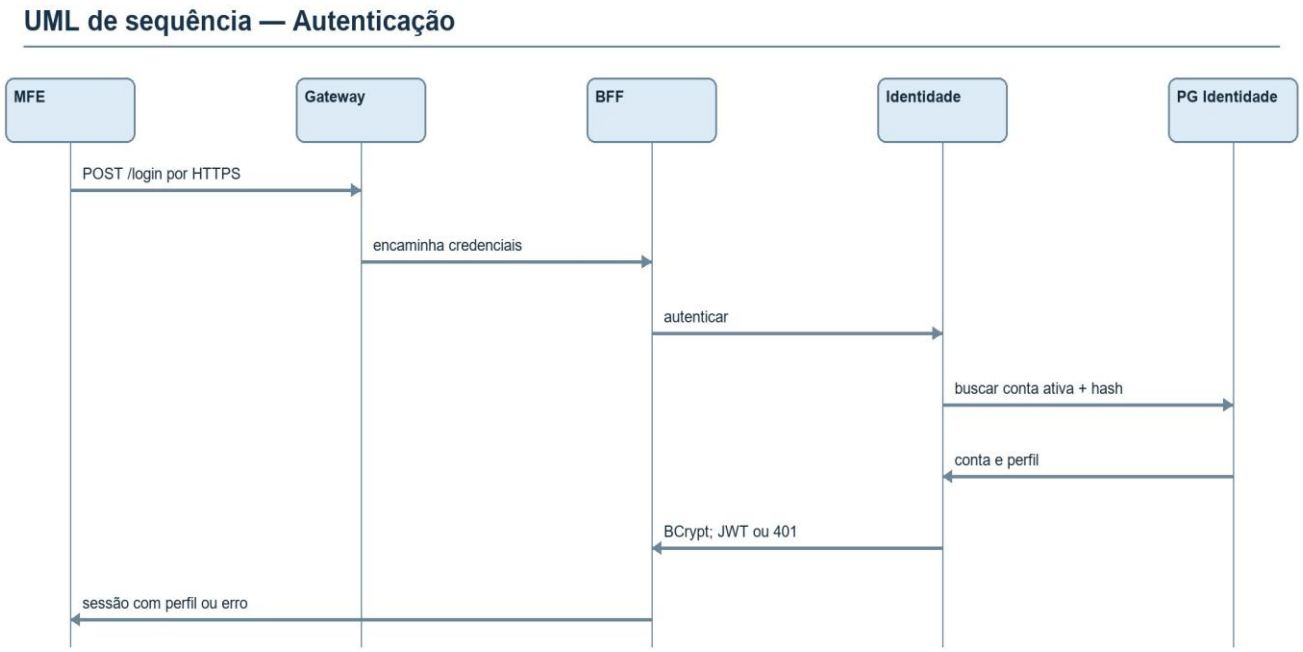


Figura 14 - UML de sequência: login, verificação BCrypt e emissão do JWT.

Sequência de autenticação: a pessoa informa e-mail/senha; o MFE envia por HTTPS; Gateway e BFF encaminham a Identidade; a conta ativa é buscada; Spring Security compara BCrypt; o JWT contém identidade/perfil, emissor, audiência e expiração; credenciais incorretas recebem 401 genérico.

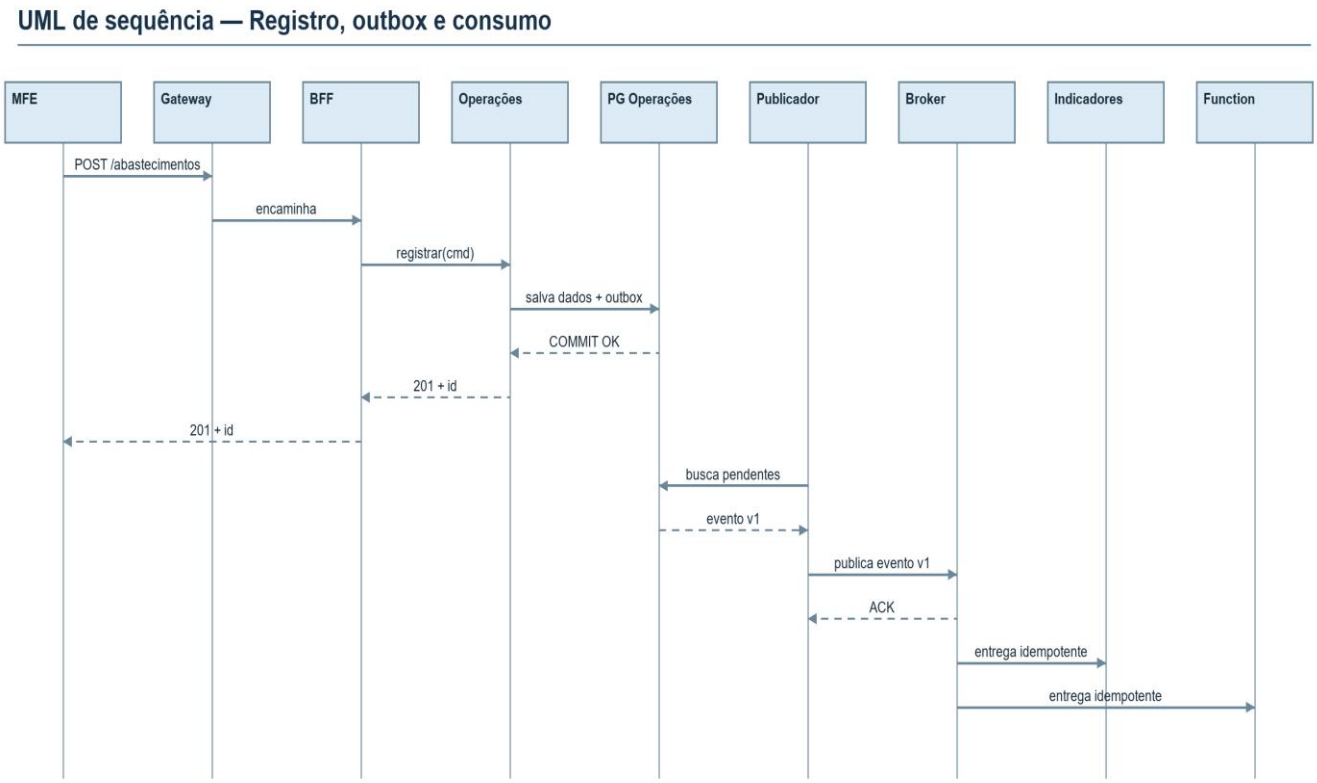


Figura 15 - UML de sequência: registro, commit local e publicação assíncrona.

Para abastecer, Operações valida os dados e referências, atualiza abastecimento, hodômetro e outbox em uma transação local. Após o commit, o publicador envia AbastecimentoRegistrado.v1 ao broker. Indicadores e a função de notificações consomem o evento de forma idempotente. Falhas transitórias provocam retentativas; falhas persistentes seguem para fila de erro.

O retorno 201 confirma a gravação em Operações, não a atualização instantânea de todas as projeções. O BFF pode sinalizar ao usuário que o painel será atualizado após o processamento do evento. Não há transação distribuída entre bancos.



Figura 16 - UML de sequência: quilometragem inferior rejeitada sem gravação.

No fluxo de rejeição, a quilometragem informada abaixo da atual gera 422 antes de qualquer escrita. Se uma exceção ocorrer após a primeira escrita da transação, o banco de Operações desfaz abastecimento, hodômetro e outbox juntos. Já uma falha posterior do broker não desfaz a operação concluída; a publicação fica pendente no outbox.

16.6.1 - Manutenção, painel e erros

Registrar manutenção valida veículo, tipo, data, km, descrição, oficina e valor. Uma próxima manutenção deve ter data posterior à atual. Operações grava histórico e outbox; Indicadores atualiza custos e disponibilidade por evento. O painel não calcula totais consultando diretamente bases de Operações ou Frota. Veículos em manutenção não contam como disponíveis, conforme a regra RB16.

Situação	Resposta	Proteção
Sem token válido	401 Unauthorized	Bloqueio antes do caso de uso.
Perfil sem permissão	403 Forbidden	Autorização no serviço, mesmo se a tela for contornada.
Recurso inexistente	404 Not Found	Nenhuma gravação iniciada.
E-mail, placa, CPF ou CNH duplicados	409 Conflict	Regra e índice UNIQUE no banco proprietário.
Regra de negócio inválida	422 Unprocessable Entity	Campos e mensagem claros, sem efeito parcial.
Falha na transação local	Erro controlado + rollback	Nenhum dado parcial e log com correlationId.
Broker indisponível após commit	201 + evento pendente	Outbox reenvia; métrica/alerta acompanham atraso.

Ponto observado no fluxo	Sinal de operação saudável

Registro local	201 apenas após commit de abastecimento, hodômetro e outbox.
Publicação	Métrica de eventos pendentes no outbox e idade do mais antigo.
Consumo	eventId registrado; duplicata não produz novo efeito; DLQ monitorada.
Interface	Confirmação do registro separada do estado eventual do painel.

16.7 - Visão de implantação

A instalação local é reproduzível e não depende de contratar recursos pagos. Cada serviço mantém configuração, credenciais, migrações e verificação de saúde próprias. A correspondência abaixo orienta uma publicação futura; não representa infraestrutura já provisionada.

Local em Docker Compose	**Publicação futura Azure**
Arquivos do shell e MFEs	Azure Static Web Apps.
Gateway e BFF	API Management e Container Apps.
Quatro serviços Java	Quatro aplicações Container Apps independentes.
PostgreSQL com quatro bases e usuários isolados	Bases PostgreSQL isoladas por serviço.
RabbitMQ e runtime local de Azure Functions	Azure Service Bus e Azure Functions.
Logs em stdout / métricas	Azure Monitor e Application Insights.

Implantação — Local e Azure

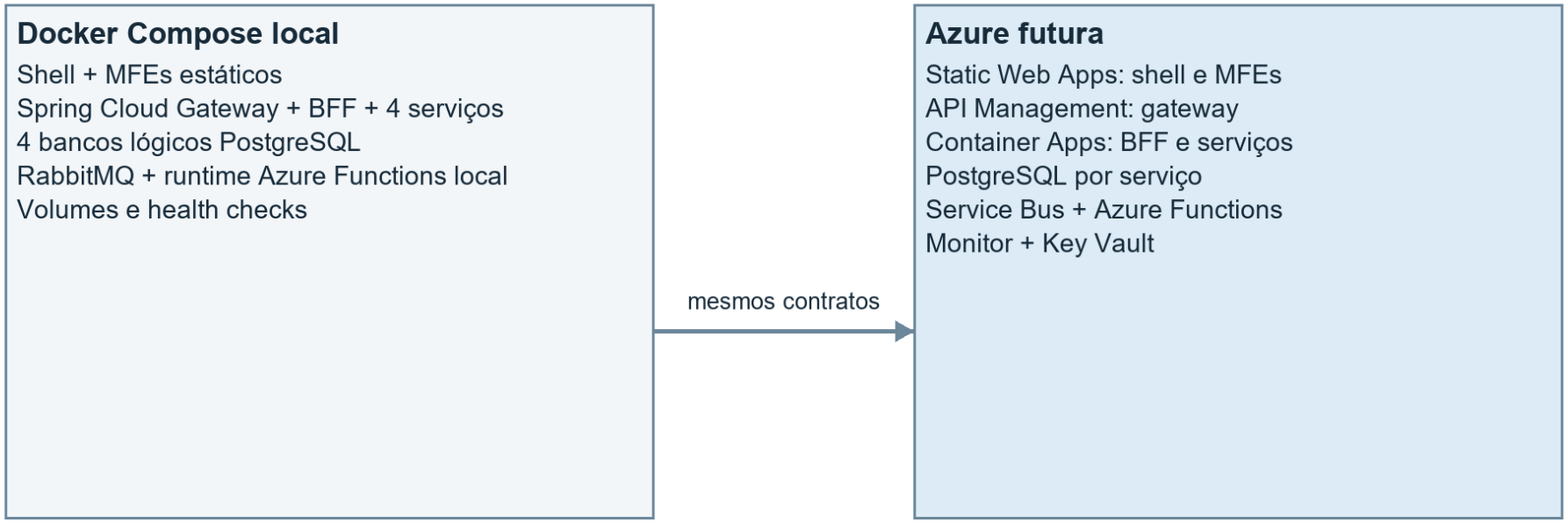


Figura 17 - Implantação local e correspondência planejada na Azure.

Cada serviço recebe configuração externa, credenciais próprias e health check. Migrações Flyway são executadas para seu banco. A publicação cloud só ocorrerá após definição de custos, segredos, backup e autorização da equipe.

Etapa operacional	Ação e verificação
Inicialização	Health check de prontidão falha claramente se banco ou broker obrigatório estiver indisponível.
Configuração	Segredos e strings de conexão em variáveis/gerenciador de segredos; nunca no Git.
Migração	Flyway de cada serviço executa apenas na sua base; versionamento imutável.
Observabilidade	Actuator, logs estruturados UTC, correlationId, métricas de outbox, lag e DLQ.
Recuperação	Backup e restauração ensaiados; verificar RPO/RTO antes de homologar Azure.
Custos	Orçamento, alertas e limites aprovados antes de provisionar recursos pagos.

16.8 - Conceitos transversais

16.8.1 - Segurança e contratos

Spring Security protege APIs; BCrypt armazena senhas; JWT contém emissor, audiência, expiração e perfil validados. Segredos ficam fora do repositório. APIs usam JSON, DTOs, paginação, identificador de correlação e erros padronizados, sem expor entidades JPA ou dados pessoais completos em logs.

Teste automatizado planejado	Violação detectada
ArchUnit: domínio não depende de Spring/JPA/web	Regra de negócio acoplada à infraestrutura.
ArchUnit: dependências seguem api → application → domain	Controller acessando repositório concreto ou domínio importando API.
ArchUnit: serviço não importa pacotes internos de outro serviço	Acoplamento direto entre microsserviços.
Conceito	**Regra transversal**
Segurança e privacidade	Autenticação, autorização por perfil, HTTPS, JWT curto, BCrypt, CPF/CNH mascarados.
Validação	Bean Validation no contrato e invariantes no domínio, inclusive em chamadas não HTTP.
Consistência	Transação por caso de uso; índices únicos; sem exclusão física destrutiva de históricos.
Rastreabilidade	Usuário responsável, timestamp UTC e correlationId em operações e eventos.
API	/api/v1, recursos no plural, JSON camelCase, datas ISO 8601, paginação e erro uniforme.
Persistência	UUID, snake_case, decimal para dinheiro/litros, Flyway por serviço e sem SQL cruzado.
Interface	Português do Brasil; estados de carregamento, vazio e erro; teclado e responsividade.
Observabilidade	Health checks, métricas, logs e traces sem senhas, tokens ou documentos completos.

16.8.2 - Eventos e integridade

O evento contém eventId, occurredAt, version, correlationId, tipo, IDs de referência e dados mínimos. Outbox persiste a intenção de publicar na transação do serviço de origem. O consumidor registra eventId processado e ignora duplicatas. Contratos são versionados; retentativas usam backoff e mensagens irrecuperáveis seguem para dead-letter queue.

Tópicos mínimos: Frota atualiza cadastros/vínculos; Operações publica AbastecimentoRegistrado.v1 e ManutencaoRegistrada.v1; Indicadores e

NotificarAbastecimento consomem apenas os campos necessários. O publicador pode reenviar; portanto, entrega “ao menos uma vez” é tratada por idempotência e não por suposição de envio único. Se a API de Frota estiver indisponível durante um comando crítico, Operações falha de forma controlada e não grava o lançamento; projeção local é usada apenas para leitura.

16.8.3 - Testes unitários de arquitetura

Teste de fronteira do frontend	MFE importando código interno de outro MFE ou chamando API sem gateway.
Teste de contrato e banco	Serviço acessando esquema de outro; evento incompatível sem versão.
Regra de arquitetura	**Teste na futura implementação**
Domain não importa org.springframework, jakarta.persistence nem web.	ArchUnit aplicado a cada módulo/serviço Java.
API só chama application; não repositório concreto.	ArchUnit sobre pacotes e classes Controller.
Infrastructure implementa portas; aplicação não importa adapter.	ArchUnit sobre dependências entre camadas.
Serviços sem ciclos nem acesso a pacotes internos uns dos outros.	Build independente + ArchUnit; dependência somente em contratos.
Controllers não devolvem entidades JPA.	Teste de reflexão/ArchUnit das assinaturas públicas.
Identidade e Frota não dependem de Operações.	ArchUnit e verificação do grafo de módulos.
MFE não importa src de outro MFE.	ESLint boundaries ou teste equivalente no CI.
Nenhuma base de outro serviço é acessada.	Credenciais isoladas + teste de integração por serviço.

16.9 - Decisões de arquitetura

ADR é um registro curto de decisão arquitetural: contexto, decisão, consequências, alternativas e estado. Cada mudança importante deve gerar nova ADR, preservando o histórico.

ADR	Estado	Decisão e consequência
ADR-001	Aceita	Microfrontends por domínio; implantação independente exige contrato e integração do shell.
ADR-002	Aceita	API Gateway + BFF; centraliza entrada e adapta respostas sem duplicar regras.
ADR-003	Aceita	Microsserviços com banco por serviço; autonomia ao custo de consistência eventual.
ADR-004	Aceita	EDA com outbox, idempotência e DLQ; entrega confiável exige operação do broker.
ADR-005	Aceita	Função serverless para notificações; desacoplamento e escala sob demanda.
ADR-006	Aceita	Clean Architecture + Vertical Slice em cada serviço; fronteiras verificadas por testes.
ADR-007	Aceita	Docker Compose local e mapeamento Azure; publicar após validar custos e segurança.

ADR-001 - Microfrontends

Campo	Registro

Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	Áreas de frota, operações e indicadores evoluirão por incrementos; a atividade exige microfrontend.
Decisão	Shell React/TypeScript/Vite compõe MFEs de domínio por rota e contrato público; design system compartilhado.
Consequências	Deploy independente e fronteiras claras; exige disciplina de versões, integração e coerência visual.
Alternativa avaliada	SPA única, mais simples, não atende ao estilo exigido.

ADR-002 - Gateway e BFF

Campo	**Registro**
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	A interface precisa de entrada controlada e respostas compostas sem expor APIs internas.
Decisão	API Gateway centraliza borda; BFF Web agrega chamadas para as telas. Regras de negócio ficam nos serviços.
Consequências	Políticas comuns e interface desacoplada; novos saltos exigem timeouts, métricas e tratamento de falhas.
Alternativa avaliada	MFE chamar todos os serviços diretamente, aumentando acoplamento e superfície pública.

ADR-003 - Microsserviços e banco por serviço

Campo	**Registro**
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	O enunciado exige implantação independente e isolamento de dados; o modelo anterior era monólito.
Decisão	Identidade, Frota, Operações e Indicadores têm deploy, PostgreSQL, credenciais e Flyway próprios.
Consequências	Autonomia e propriedade explícita; consultas entre domínios exigem APIs/eventos e consistência eventual.
Alternativa avaliada	Monólito modular com banco único, descartado para esta entrega.

ADR-004 - EDA com outbox

Campo	**Registro**
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	Painel e notificações devem reagir a fatos sem fazer parte da transação crítica.
Decisão	Publicar fatos via outbox e broker; consumidores idempotentes, contratos versionados, retry e DLQ.
Consequências	Menor acoplamento temporal; operação do broker e monitoramento de lag tornam-se obrigatórios.
Alternativa avaliada	Chamadas síncronas em cascata; poderiam fazer o registro depender da notificação.

ADR-005 - Serverless para notificação

Campo	**Registro**

Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	Envio de aviso é eventual e disparado por evento, não uma regra transacional.
Decisão	Azure Function NotificarAbastecimento consome evento local/Azure e chama provedor de e-mail com idempotência.
Consequências	Escala por demanda; validar tempo de inicialização, timeout, repetição e custo.
Alternativa avaliada	Worker Java sempre ligado; adicionaria custo e não demonstraria serverless.

ADR-006 - Clean Architecture e Vertical Slice

Campo	Registro
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	Regras de km, vínculos e valores não devem depender do framework; mudanças devem ficar localizadas.
Decisão	Pacotes por caso de uso, com api/application/domain/infrastructure e dependências voltadas para dentro.
Consequências	Testabilidade e fronteiras explícitas; mais interfaces e testes ArchUnit necessários.
Alternativa avaliada	Camadas globais Controller/Service/Repository que dispersam uma funcionalidade.

ADR-007 - Compose local e Azure futura

Campo	Registro
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	A solução precisa ser demonstrável sem custo cloud, mas manter plano de publicação.
Decisão	Compose sobe shell/MFEs, gateway, BFF, serviços, bancos, broker e runtime de função; Azure mapeia esses blocos.
Consequências	Ambiente reproduzível; diferenças de serviços gerenciados exigem testes de contrato e homologação.
Alternativa avaliada	Provisionar cloud antes de validar custos, segredos e backups.

ADR-008 - Permissões e segurança

Campo	Registro
Estado	Aceita como arquitetura-alvo; implementação e testes pendentes.
Contexto	Administrador gerencia usuários; Gestor executa operações; dados pessoais precisam de proteção.
Decisão	Spring Security, JWT curto e BCrypt; autorização aplicada em cada serviço, não apenas na interface.
Consequências	Defesa em profundidade e auditoria; chaves, expiração e perfis devem ser governados.
Alternativa avaliada	Permissão apenas no MFE, insuficiente contra chamadas diretas.

Nova ADR é necessária quando mudar versão principal, fronteira de serviço, formato de evento, provedor pago, regra de autorização ou decisão com efeito relevante sobre segurança, consistência e custo. O histórico de ADRs substituídas não deve ser apagado.

16.10 - Requisitos de qualidade

Árvore de qualidade: Segurança → autenticação, autorização, privacidade; Consistência/Rastreabilidade → invariantes, transação local, histórico e responsável; Usabilidade → formulários e acessibilidade; Operabilidade → saúde, lag, logs e recuperação; Manutenibilidade → fronteiras, migrações e testes. A tabela transforma cada ramo em cenário verificável.

ID	Categoria	Estímulo	Evidência de aceitação
CQ01	Segurança	Pessoa sem sessão acessa API protegida.	401; caso de uso não executado.
CQ02	Autorização	Gestor tenta alterar usuários/perfis.	403 mesmo contornando a interface.
CQ03	Consistência	Abastecimento traz km inferior ao atual.	422; nenhum abastecimento/hodômetro/outbox gravado.
CQ04	Atomicidade	Escrita falha depois de iniciar abastecimento.	Rollback local integral no banco Operações.
CQ05	Histórico	Consultar operações de veículo desativado.	Registros preservados com data e responsável.
CQ06	Usabilidade	Formulário tem campos inválidos.	Dados válidos mantidos; campos marcados e
			mensagem compreensível.
CQ07	Operabilidade	Banco indisponível em health check.	Prontidão falha; log correlacionado sem segredo.
CQ08	Manutenibilidade	Regra de abastecimento muda.	Mudança fica na fatia e testes detectam regressão.
CQ09	Desempenho	100 usuários consultam listas paginadas.	p95 < 2 s no ambiente de homologação definido.
CQ10	Recuperação	Falha lógica exige restauração.	RPO ≤ 24 h; RTO ≤ 4 h em ensaio documentado.
CQ11	Arquitetura	Fatia importa classe interna alheia.	Teste de arquitetura falha no CI.
CQ12	EDA	Evento chega duas vezes.	Consumidor mantém um único efeito por eventId.

Critérios extras da arquitetura distribuída: interrupção do broker não cancela uma transação confirmada; outbox pendente é reprocessado; função serverless tolera repetição e timeout; falha de um MFE não impede o shell de servir áreas independentes. Essas condições exigem testes de falha além dos cenários CQ01–CQ12.

16.11 - Riscos e dívidas técnicas

Prioridade	Risco / dívida	Impacto	Tratamento
Alta	Abastecimentos simultâneos atualizam o mesmo hodômetro.	Perda de atualização.	Versão/lock transacional e teste concorrente PostgreSQL.

Alta	Retroativos, correções e cancelamentos sem política.	Histórico incoerente.	Decidir por ADR antes de editar/cancelar; preservar auditoria.
Alta	Exposição de CPF, CNH, senha ou JWT.	Privacidade/conta comprometida.	Mascaramento, logs mínimos, BCrypt, rotação de segredos.
Alta	Evento duplicado/perdido.	Painel e avisos incorretos.	Outbox, idempotência, retry, DLQ e métricas de lag.
Alta	Serviços distribuídos com contratos instáveis.	Retrabalho e falhas de integração.	Contratos mínimos, teste de contrato e ADR por fronteira.
Alta	Status de veículo/CNH muda entre validação e gravação.	Operação aceita com referência obsoleta.	Contrato síncrono de Frota, versão de estado e reconciliação; fechar política em ADR.
Média	CNH vencida ou vínculos simultâneos.	Divergência de regras.	Fechar decisão antes da fatia de vínculo e testar domínio.
Média	Dashboard sem índices/projeções adequadas.	Consulta lenta.	Medir p95 e criar índices por período, veículo e status.
Média	Gateway indisponível/mal configurado.	Entrada das APIs interrompida.	Health check, configuração versionada e teste de contingência.
Média	Cold start ou repetição da função.	Atraso/aviso duplicado.	Timeout, métricas e handler idempotente.
Média	Azure com custo/segredos incorretos.	Custo inesperado ou falha.	Orçamento e aprovação antes de provisionar; Key Vault.
Média	Cobertura E2E incompleta.	Regressão entre MFE/BFF/API.	Um fluxo E2E por fatia crítica.
Média	C4/DER divergirem do código.	Documento perde valor.	Revisão dos artefatos a cada sprint e teste de fronteira.
Baixa	Microfrontends fragmentam UI.	Experiência inconsistente.	Design system compartilhado e teste visual.
Baixa	Comprovantes fora do MVP.	Sem anexos iniciais.	Manter Blob Storage no backlog, sem código prematuro.

17 - Plano de implementação por sprints

Sprint	Entrega principal	Prova de conclusão
0 - Fundamentos	Repositório, contratos, Docker Compose, gateway, bases isoladas e CI.	Ambiente sobe localmente; teste arquitetural inicial passa.
1 - Identidade e Frota	JWT/BCrypt, usuários, veículos, motoristas e microfrontends básicos.	Login, autorização e cadastros testados.
2 - Operações e EDA	Abastecimento, hodômetro, manutenção, outbox e broker.	Transação local, retentativa e idempotência demonstradas.

3 - Leitura e serverless	Indicadores, dashboard e função de notificações.	Projeções atualizam por eventos; falhas vão para DLQ.
4 - Qualidade e publicação	Testes de arquitetura/contrato, observabilidade e plano Azure.	C4/UML/DER batem com o código; custos e segredos revisados.

Fundamento	O que é / por que usar	Quando a equipe implementa
API REST e DTO	Contrato HTTP estável sem expor objetos de persistência.	Primeiro caso de uso da Sprint 1.
Flyway e DB por serviço	Histórico reprodutível e propriedade dos dados.	Estrutura da Sprint 0; tabelas por fatia.
JWT + BCrypt	Sessão autenticada e senha protegida.	Identidade na Sprint 1.
Clean + Vertical Slice	Fronteiras de dependência e código por caso de uso.	Desde a primeira fatia Java.
Outbox e EDA	Publicação segura após transação sem acoplar consumidores.	Operações na Sprint 2.
Serverless	Código acionado por evento, escalado pelo provedor.	Notificação na Sprint 3.
ArchUnit	Teste automatizado das decisões arquiteturais.	Regra inicial na Sprint 0; ampliar com cada serviço.

18 - Glossário

Termo	Definição no NexFleet
ADR	Registro de decisão com contexto, estado, alternativas e consequências.
API Gateway	Ponto público de entrada com roteamento e políticas comuns.
BFF	Backend específico da interface web, responsável por compor respostas.
C4 Model	Visões de contexto, contêineres, componentes e código.
Clean Architecture	Regra de dependências voltadas ao domínio, longe dos detalhes externos.
Database per Service	Cada serviço possui seu armazenamento, credenciais e migrações.
EDA	Comunicação por eventos de fatos ocorridos.
Microfrontend	Parte da interface com autonomia de construção e publicação.
Outbox	Registro transacional de dado de negócio e evento ainda não publicado.
Serverless	Execução de função acionada por evento em plataforma gerenciada.
Vertical Slice	Organização do código por fluxo/caso de uso completo.

19 - Critérios de entrega e revisão

Artefato/estilo	Evidência exigida na entrega do código
C4 níveis 1–4, UML e DER	Diagramas coincidem com processos, pacotes, classes, chamadas e migrações Flyway; sem FK cruzada.

Microfrontends, Gateway e BFF	Shell e três MFEs publicáveis separadamente; rotas, políticas e composição testadas.
EDA e serverless	Broker, outbox, consumidor idempotente, DLQ e função disparada por evento.
Clean, Vertical Slice e testes de arquitetura	Domínio sem framework; portas/adaptadores, ArchUnit e fronteiras MFE verificados no CI.
Segurança, qualidade e identificação	JWT/BCrypt, provas CQ01–CQ12 e quatro integrantes na capa do PDF.

Referências

arc42: https://arc42.org/overview/ | template em português: https://arc42.org/download/
C4 Model: https://c4model.com/diagrams | Software Architecture Canvas: (workingsoftware.dev).
ArchUnit: https://www.archunit.org/userguide/html/000_Index.html